

El kernel Gabor como átomo

envolvente $\times$ onda: qué es, y qué le hace a cada fase de la optimización

Proyecto 2DGS $\rightarrow$ Beta / Gabor Splatting

31 de julio de 2026

Este documento describe el kernel implementado en la rama gabor-atomo-envolvente-onda (commit 54274f5 y posteriores) y sigue su rastro por todas las fases del entrenamiento: preprocesado, forward, backward, proyección en Python, densificación y serialización. Cada fórmula corresponde a una línea concreta del código, citada como `archivo:línea`. Las cifras experimentales son las de los runs 5, 6 y 7 sobre *flowers*.

Índice

1. El problema que resuelve	4
1.1. El kernel anterior y su punto de degeneración	4
1.2. La observación de fondo	4
1.3. La solución: factorizar	5
2. Definición formal	6
2.1. Los tres modos	6

2.2.	La frecuencia: el modo world y por qué importa	6
2.3.	La cota del presupuesto	7
2.4.	Cómo se lee el papel de γ	8
3.	El reparto Python / CUDA	8
3.1.	El tensor de cinco columnas	8
3.2.	Dónde vive cada símbolo	9
4.	Fase 1 — Preprocess	9
5.	Fase 2 — Forward	10
5.1.	Antes del kernel: la geometría	10
5.2.	El interruptor de escala	11
5.3.	La evaluación	11
5.4.	Coste	12
6.	Fase 3 — Backward	13
6.1.	Regla número uno: recomputar, no recordar	13
6.2.	El punto de partida: $\partial L / \partial \alpha$	13
6.3.	Los cinco gradientes del bloque Gabor	14
6.4.	Por qué los clamps son distintos	15
6.5.	El gate $\beta \geq 0,1$ ya no cubre todo	15
6.6.	La bifurcación radial / direccional en la cadena de ρ	15
6.7.	El guard r_{safe}	16
6.8.	Una asimetría real que conviene conocer	16

7. Fase 4 — Python: restricciones y paso del optimizador	17
7.1. La proyección: gradiente proyectado exacto	17
7.2. La doble imposición de $a_n \geq 0$	18
7.3. La autocalibración de f_1	18
7.4. Tasa de aprendizaje	19
8. Fase 5 — Densificación y ciclo de vida	19
8.1. Herencia en clone y split	19
8.2. Pero la frecuencia sí se hereda mal (y esto es un hallazgo) . .	20
8.3. Serialización	21
9. Verificación	21
10. Qué se midió	22
10.1. El rediseño funciona; la onda no compra	22
10.2. La firma que sí se repite	22
11. Invariantes	23
A. Símbolos	24
B. Mapa del código	24
C. Documentos relacionados	25

1. El problema que resuelve

1.1. El kernel anterior y su punto de degeneración

Hasta el run 4 el kernel de opacidad era una serie de cosenos elevada a β :

$$K_{\text{legacy}}(r) = \left(\frac{f(r)}{f(0)} \right)^\beta, \quad f(r) = \frac{1}{2} + \sum_{n=1}^3 a_n \cos((2n-1)\pi r),$$

con $r = \sqrt{\rho}$ la distancia normalizada al centro del surfel y $a_n \geq 0$, $\sum a_n \leq \frac{1}{2}$ impuesto por proyección.

Tiene un defecto estructural que no se ve mirando la fórmula, sino preguntándose *qué kernel sale cuando el modelo no aprende nada*. Con $a = 0$ queda $f \equiv \frac{1}{2}$, es decir

$$K_{\text{legacy}}|_{a=0} \equiv 1 \quad \text{en todo el soporte,}$$

un **disco plano de opacidad uniforme**: el peor kernel posible. Y ese punto no está en un rincón cualquiera del espacio de parámetros: está *en la frontera* $a_n \geq 0$, justo donde el optimizador empuja cuando quiere apagar la modulación.

Peor aún: el kernel de referencia, la *tent* $(1-r)^\beta$ del run 67, se obtiene con $a = (4/\pi^2)(1, \frac{1}{9}, \frac{1}{25})$ renormalizado, lo que consume $\Sigma a = 0,4665$, el **93,3 %** del presupuesto disponible $\Sigma a \leq \frac{1}{2}$. Dicho de otro modo: en el kernel legacy, *limitarse a reproducir la envolvente ya agota casi todo el presupuesto*, y lo que queda para modular la forma es una miseria.

1.2. La observación de fondo

Un átomo de Gabor es, por definición, un producto de dos factores independientes: una **envolvente** que decide hasta dónde llega el soporte, y una **onda** que decide cómo oscila dentro de él. El kernel legacy no tiene esa separación: el decaimiento y la oscilación salen los dos del mismo $\sum a_n \cos(\cdot)$, así que los tres coeficientes hacen dos trabajos con un solo presupuesto.

La medida que lo confirma: solo el **7,9 %** del conjunto factible $\{a_n \geq 0, \Sigma a \leq \frac{1}{2}\}$ da un perfil decreciente. El otro 92 % son kernels con lóbulos secundarios — los anillos concéntricos que se veían en el césped.

1.3. La solución: factorizar

$$K(r, t) = \underbrace{(1 - r)^\beta}_{\text{envolvente } E} \cdot \underbrace{S(\varphi t)}_{\text{onda}} \quad (1)$$

$$S(x) = b + \sum_{n=1}^3 a_n \cos((2n - 1)x), \quad (2)$$

$$b = \gamma + (1 - \gamma)(1 - \Sigma a), \quad \Sigma a = \sum_n a_n. \quad (3)$$

El término b es el **pedestal**, tomado de AdaGaR. Es la pieza que lo cambia todo, porque hace que

$$a = 0 \implies b = 1 \implies S \equiv 1 \implies K = (1 - r)^\beta,$$

la *tent del run 67* exactamente. Y el modelo se inicializa ahí (`gaussian_model.py:298, a_init_row`). Las consecuencias no son cosméticas:

- El **baseline es el punto neutro** del espacio de parámetros, no una esquina. Aprender la forma solo puede sumar; si no encuentra nada, se queda donde estaba.
- La envolvente es **gratis**: ya no se paga con el presupuesto de a . Los tres coeficientes quedan libres para lo único que deben hacer.
- La fracción de kernels decrecientes dentro del conjunto factible sube de 7,9 % a **70,8 %** (Monte Carlo convergido).
- El control experimental es exacto: las primeras iteraciones deben reproducir el run 67 dígito a dígito.

2. Definición formal

2.1. Los tres modos

El modo lo fija la variable de entorno GABOR_KERNEL y viaja a CUDA como el entero gabor_mode (gabor_kernel.h:47-49):

modo	id	kernel
legacy	0	$(f/f_0)^\beta$
radial	1	$(1-r)^\beta S(\varphi r)$
dir	2	$(1-r)^\beta S(\varphi u)$

La diferencia entre radial y dir es el argumento de la onda:

- **radial**: $t = r$, la modulación es concéntrica. Produce *anillos*.
- **dir**: $t = u$, la primera coordenada local del surfel. La modulación son *franja*s paralelas en una dirección, que es lo que un átomo de Gabor es de verdad. La orientación de las franjas la hereda de la rotación del surfel, así que **ya es aprendible** sin añadir ningún parámetro.

Los runs 5, 6 y 7 usan dir, que en el A/B local ganó con *la mitad de splats* que radial (287k contra 597k): más expresividad por primitiva, que es la promesa del átomo.

2.2. La frecuencia: el modo world y por qué importa

φ es adimensional dentro de CUDA, pero se construye en Python (gaussian_model.py:360-371) de dos formas:

$$\text{norm: } \varphi = \pi \tag{4}$$

$$\text{world: } \varphi = \frac{f_1}{\text{extent}} s_u \tag{5}$$

donde s_u es la escala del eje u en unidades de mundo y *extent* es `spatial_lr_scale`.

La consecuencia de (5) merece detenerse. La coordenada u que recibe CUDA está *normalizada por la escala del splat* (por eso el soporte es $\rho < 1$), o sea $u = x_{\text{mundo}}/s_u$. Al multiplicar:

$$\varphi u = \frac{f_1 s_u}{\text{extent}} \cdot \frac{x_{\text{mundo}}}{s_u} = \frac{f_1}{\text{extent}} x_{\text{mundo}}.$$

Las escalas **se cancelan**. La longitud de onda en el mundo es

$$\lambda = \frac{2\pi \text{ extent}}{f_1},$$

la misma para todos los splats. El modo `world` impone una rejilla de frecuencia fija en la escena, y cada surfel muestra tantos ciclos como le quepan según su tamaño. Un splat grande desarrolla textura; uno pequeño no llega ni a medio ciclo y degenera al tent.

En modo `norm` pasa lo contrario: $\varphi = \pi$ significa medio periodo dentro del soporte *sea cual sea el tamaño*, es decir la frecuencia va atada al splat y no a la escena.

2.3. La cota del presupuesto

El requisito es $S \geq 0$: la onda no debe volverse negativa dentro del soporte, o el kernel pintaría opacidad negativa (que el forward recorta, pero recortar es perder gradiente). El peor caso de (2) es que los tres cosenos valgan -1 a la vez, así que

$$S_{\min} = b - \Sigma a = \gamma + (1 - \gamma)(1 - \Sigma a) - \Sigma a = 1 - (2 - \gamma) \Sigma a,$$

y por tanto

$$\boxed{\Sigma a \leq \frac{1}{2 - \gamma}} \tag{6}$$

Con $\gamma = 0,3$ salen 0,5882 (gaussian_model.py:309–320). Nótese que la cota es *más generosa* que el $\frac{1}{2}$ del legacy: es el mismo razonamiento, pero teniendo en cuenta que el pedestal sube automáticamente cuando Σa baja.

El dial κ (GABOR_KAPPA) multiplica la cota. Con $\kappa < 1$ se acota además el contraste valle/pico a $(1 - \kappa)/(1 + \kappa)$. Se dejó en 1,0 en todos los runs.

2.4. Cómo se lee el papel de γ

γ es el suelo del pedestal. Fija cuánta modulación puede llegar a haber como máximo:

$$b \in [\gamma, 1], \quad b = 1 \text{ si } \Sigma a = 0, \quad b = \gamma \text{ si } \Sigma a = 1.$$

Con $\gamma = 0$ el pedestal podría anularse del todo y la envolvente volvería a depender de a ; con $\gamma = 1$ el pedestal sería constante y la cota (6) se quedaría en 1. El 0,3 es el valor de AdaGaR.

3. El reparto Python / CUDA

Esta es la decisión de ingeniería con más consecuencias prácticas del rediseño, y no se ve en las ecuaciones.

3.1. El tensor de cinco columnas

El rasterizer recibe, por Gaussiana, un bloque de **5 floats** (gabor_kernel.h:45, GABOR_STRIDE 5):

$$[a_1, a_2, a_3, \varphi, b]$$

φ y b **no son parámetros entrenables**. Son funciones diferenciables de a y de las escalas, calculadas en Python (get_gabor, gaussian_model.py:352–374). CUDA las trata como entradas independientes, devuelve $\partial L / \partial \varphi$ y $\partial L / \partial b$, y es **autograd** quien compone la cadena hacia $_a$ y $_scaling$.

Las tres razones, en orden de importancia:

1. γ y f_1 **no viven en CUDA**. Barrerlos no recompila nada. Todo el sweep de GABOR_F1 (runs 6 y 7) se hizo con una variable de entorno.
2. **El gradiente “crece y ganarás textura” llega solo**. Como φ depende de s_u por (5), autograd propaga $\partial L / \partial \varphi \rightarrow \partial L / \partial s_u$ automáticamente. Ese camino *no existe* en el rasterizer: es puro Python.
3. **Una sola fórmula**. El pedestal se escribe una vez.

El coste: cambiar a de $(N, 3)$ a $(N, 5)$ **obliga a recompilar el rasterizer**. Es el único cambio del rediseño que lo exige.

3.2. Dónde vive cada símbolo

símbolo	qué es	dónde
a_1, a_2, a_3	entrenable	Python + CUDA
β	entrenable	Python + CUDA
s_u, s_v	entrenable	Python + CUDA
φ	derivado	Python
b	derivado	Python
γ	constante	Python
f_1	constante	Python
κ	constante	Python

Solo tres familias de parámetros del kernel se entrenan: a , β y, por la vía indirecta de φ , las escalas.

4. Fase 1 — Preprocess

El kernel preprocessCUDA corre **una vez por Gaussiana** (no por píxel) y hace: culling de frustum, matriz de transformación, radio en pantalla, tiles tocados, color desde armónicos esféricos.

Del bloque Gabor solo hace una cosa (forward.cu:193–196):

```
#pragma unroll
for (int _k = 0; _k < GABOR_STRIDE; _k++)
    a_out[GABOR_STRIDE*idx + _k] = a_in[GABOR_STRIDE*idx + _k];
```

Copiarlo al buffer de geometría, igual que β . Y esto tiene una consecuencia que conviene tener presente:

El binning no sabe nada del kernel. El radio en pantalla, la caja envolvente y el conjunto de tiles se calculan *solo* con la geometría (μ , escalas, rotación). La onda no ensancha ni estrecha la huella.

Es correcto, porque el soporte lo fija la envolvente $(1 - r)^\beta$, que se anula en $r = 1$ pase lo que pase con S . Pero significa que el coste de memoria del rediseño está aquí: el buffer de geometría pasa de 3 a 5 floats por splat. Con 4,9 M splats son ~ 39 MB extra, despreciable frente a lo que ocupa el modelo.

5. Fase 2 — Forward

Aquí es donde se evalúa el kernel: **una vez por pareja (píxel, splat)**, que es el bucle más caliente de todo el sistema.

5.1. Antes del kernel: la geometría

Cada hilo intersecta el rayo del píxel con el plano del surfel y obtiene las coordenadas locales $s = (u, v)$ (forward.cu:418–434):

$$\rho_{3d} = u^2 + v^2 \quad \text{geometría real} \quad (7)$$

$$\rho_{2d} = \text{FilterInvSquare} \parallel \mathbf{d} \parallel^2 \quad \text{filtro paso-bajo} \quad (8)$$

$$\rho = \min(\rho_{3d}, \rho_{2d}), \quad r = \sqrt{\rho} \quad (9)$$

El min es del 2DGS original: cuando el splat se hace sub-píxel, quien manda es el filtro de pantalla, no la geometría. Y hay un guard, `if (r2 >= 1.0f)`

continúe, que garantiza $r < 1$ en todo lo que sigue — un dato del que dependen varias fórmulas del backward.

5.2. El interruptor de escala

```
const bool modulate = (rho3d <= rho2d);
const float t = (gabor_mode == GABOR_MODE_DIR) ? s.x : r;
const float S = modulate
    ? gabor_wave(a1,a2,a3,b,phi,t,dS_dt,dS_dphi)
    : 1.0f;
```

La onda solo se aplica donde manda la geometría real. Si el splat es sub-píxel, S se fuerza a 1 y el kernel es tent puro. El razonamiento:

- Un splat sub-píxel *no puede mostrar textura*. Modularlo sería pintar detalle por debajo de la frecuencia de Nyquist de la pantalla: aliasing, no señal.
- Mantiene intacto el gradiente de la rama ρ_{2d} , que es puramente geométrico. Sin el interruptor aparecerían términos cruzados entre la onda y el filtro paso-bajo.

Este interruptor es también el origen del diagnóstico [GABOR-W]: es la frontera entre “la onda está encendida” y “este splat es un tent”.

5.3. La evaluación

```
float dE_dr;
const float E = gabor_envelope(r, beta_j, dE_dr);
kernel = fmaxf(E * S, 0.0f); // S<0 -> el lobulo negativo no pinta
float alpha = min(0.99f, opa * kernel);
if (alpha < 1.0f/255.0f) continue;
```

Con la envolvente (gabor_kernel.h:70–76):

$$E = \left(\max(1 - r, 10^{-6}) \right)^{\beta}, \quad \frac{\partial E}{\partial r} = -\beta (1 - r)^{\beta-1}$$

El piso 10^{-6} evita $\text{pow}(0, \beta)$ con $\beta < 1$ en el borde.

Y la onda (`gabor_kernel.h:55-65`), que calcula de una vez el valor y las dos derivadas que hará falta más tarde:

$$S = b + \sum_n a_n \cos(w_n t), \quad w_n = (2n - 1)\varphi$$

$$\frac{\partial S}{\partial t} = - \sum_n a_n w_n \sin(w_n t), \quad \frac{\partial S}{\partial \varphi} = -t \sum_n a_n (2n - 1) \sin(w_n t)$$

Tres observaciones sobre el `fmaxf(E*S, 0)`:

1. La cota (6) ya garantiza $S \geq 0$, así que *en teoría* el `fmaxf` es código muerto. Está por lo mismo que los demás pisos del rasterizer: un `.ply` de un run viejo, o un epsilon de `float32`, no deben poder producir opacidad negativa.
2. A diferencia del legacy, aquí **no hay división**. El legacy dividía por f_0 , y esa división fue la singularidad que reventó el run 1 (52,6 M splats y NaN en la iteración 4.100).
3. $K \leq 1$ no está garantizado por construcción, pero sí en la práctica: $E \leq 1$ y $S \leq b + \Sigma a \leq 1$ cuando $\Sigma a \leq 1/(2 - \gamma)$ y $\gamma \leq 1$. El `min(0.99f, ...)` de α es un guard aparte, heredado del 3DGS.

5.4. Coste

Por pareja píxel-splat modulada: **3 senos y 3 cosenos** más un `powf`. El tent solo pagaba el `powf`. En unidades de la SFU de la GPU eso es del orden de $6 \times$ más trabajo trascendente en el punto más caliente del pipeline.

Dos atenuantes medidos: (a) el interruptor de escala deja fuera a los splats sub-píxel, que en estas escenas son mayoría; (b) los senos y cosenos se evalúan en `sinf/cosf` (precisión simple, hardware), no en las versiones de doble precisión.

Hay una optimización pendiente que *no* es del kernel pero le afecta: el cutoff sigue en 3,0 (valor gaussiano) cuando el soporte es compacto en $r = 1$, así

que se están evaluando del orden de $9 \times$ más píxeles de los necesarios. Ver el punto 3 de la lista de pendientes del proyecto.

6. Fase 3 — Backward

El backward recorre las mismas parejas píxel-splat en orden inverso, reconstruyendo la transmitancia T hacia atrás.

6.1. Regla número uno: recomputar, no recordar

El backward **vuelve a evaluar el kernel entero** en vez de guardarlo. Es lo correcto en memoria (guardar K por pareja píxel-splat sería un buffer prohibitivo) pero implica que las dos fórmulas tienen que coincidir exactamente. Este proyecto ya pagó una vez ese precio: en los runs 60 y 62 el clamp de α del backward no coincidía con el del forward y los gradientes se inflaban en todo el frente del píxel.

De ahí que la definición viva en **un único header**, `gabor_kernel.h`, que incluyen los dos ficheros. Y de ahí la línea explícita (`backward.cu:368`):

```
float alpha = min(0.99f, opa * kernel); // el MISMO clamp que el forward
const float G = kernel;                // sin clampar: dalpha/dopa
```

6.2. El punto de partida: $\partial L / \partial \alpha$

Antes de tocar el kernel, el backward acumula en `dL_dalpha` todo lo que depende de α : color, profundidad, normales, mapa de acumulación y distorsión (`backward.cu:382-436`). Al final multiplica por T .

Desde ahí salen **cinco** gradientes nuevos, más los que ya existían.

6.3. Los cinco gradientes del bloque Gabor

Todos salen de derivar (1) directamente, sin división:

$$\frac{\partial \alpha}{\partial \beta} = \text{opa} \cdot E \cdot S \cdot \ln(1 - r) \quad (10)$$

$$\frac{\partial \alpha}{\partial a_n} = \text{opa} \cdot E \cdot \cos(w_n t) \quad (11)$$

$$\frac{\partial \alpha}{\partial b} = \text{opa} \cdot E \quad (12)$$

$$\frac{\partial \alpha}{\partial \varphi} = \text{opa} \cdot E \cdot \frac{\partial S}{\partial \varphi} \quad (13)$$

Compárese con el legacy, cuyo gradiente de a_n era

$$\frac{\partial \alpha}{\partial a_n} = \frac{\text{opa} \cdot \beta}{f_0} g^{\beta-1} (c_n - g),$$

con un f_0 en el denominador, un $g^{\beta-1}$ que hay que apuntalar cuando $\beta < 1$, y una resta $(c_n - g)$ que acopla los tres coeficientes entre sí. El del átomo, (11), es un coseno multiplicado por dos factores positivos. **La simplificación del backward es, en sí misma, un resultado del rediseño:** menos casos singulares que apuntalar.

La implementación (backward.cu:466–490):

```
const float opaE = opa * E_env;
if (beta_j >= 0.1f) {
    float grad_beta = dL_dalpha * opaE * S_mod
        * logf(fmaxf(1.0f - r, 1e-6f));
    grad_beta = fminf(fmaxf(grad_beta, -1e-3f), 1e-3f);
    atomicAdd(&dL_dbeta[global_id], grad_beta);
}
if (modulate) {
    const float base = dL_dalpha * opaE;
    ga1 = clamp(base * cosf(w1*t_mod), -50, 50);
    ...
    gph = clamp(base * dS_dphi, -50, 50);
}
```

```

gb = clamp(base, -50, 50);
atomicAdd(&dL_da[global_id*GABOR_STRIDE + k], g_k);
}

```

6.4. Por qué los clamps son distintos

$\pm 10^{-3}$ para β y ± 50 para el resto. No es arbitrario:

- El de β es un **limitador de señal** heredado del kernel beta original, donde $\ln g$ podía explotar. En el átomo el logaritmo es $\ln(1 - r)$, *finito en toda la huella* porque el guard $r < 1$ lo garantiza — ya no haría falta, y se mantiene por prudencia.
- El de a , φ y b es un **backstop de NaN**, no un limitador: esos gradientes son $O(1)$ y ± 50 no los toca nunca. Es importante no confundirlos: clampar a a 10^{-3} aplastaría la señal y el kernel no aprendería nada.

6.5. El gate $\beta \geq 0,1$ ya no cubre todo

En el legacy, los splats con $\beta < 0,1$ se saltaban el gradiente de β para evitar NaN. En el átomo ese gate sigue aplicándose *solo a β* ; a , φ y b quedan siempre vivos, porque (11)–(13) no tienen ninguna singularidad. Un splat con β minúsculo puede seguir aprendiendo forma.

6.6. La bifurcación radial / direccional en la cadena de ρ

Esta es la parte del backward donde es más fácil equivocarse. La derivada respecto de la geometría (backward.cu:551–564):

```

float dalpha_dr = opa * dE_dr * S_mod;
if (modulate && gabor_mode == GABOR_MODE_RADIAL)
    dalpha_dr += opa * E_env * dS_dt;
d_alpha_d_rho = dalpha_dr / (2.0f * r_safe);
if (modulate && gabor_mode == GABOR_MODE_DIR)

```

$$\text{dalphi_dsx_extra} = \text{opa} * E_env * dS_dt;$$

El razonamiento:

- En **radial** la onda depende de r , así que su contribución entra entera por la regla de la cadena de ρ : $\partial\alpha/\partial r = \text{opa} (E'S + ES')$, y luego $\partial r/\partial\rho = 1/(2r)$.
- En **dir** la onda depende de $u = s_x$, que *no pasa por ρ* . Su término se lleva aparte y se inyecta directamente en la componente x del gradiente respecto de las coordenadas locales (`backward.cu`:593–596):

$$dL_ds.x = \underbrace{dL_d_rho \cdot 2s_x}_{\text{geometría}} + \underbrace{dL_dalphi \cdot \text{dalphi_dsx_extra}}_{\text{onda}} + dL_dz \cdot T_{w,x}$$

Si se olvidara ese término, la onda direccional sería invisible para la geometría: el modelo no podría mover ni rotar el surfel para alinear las franjas con la textura de la escena, que es justamente lo que se le pide. Nótese la asimetría: la componente y no lleva término de onda, porque en modo `dir` la onda no depende de v .

6.7. El guard r_{safe}

$\partial r/\partial\rho = 1/(2r)$ diverge en $r \rightarrow 0$. El código usa $r_{\text{safe}} = \max(r, 10^{-6})$. La divergencia no es real: en el centro $\partial\alpha/\partial r \rightarrow 0$ (tanto E' como S' contienen factores que se anulan), así que el cociente tiene límite finito; el guard solo evita el 0/0 numérico.

6.8. Una asimetría real que conviene conocer

El término de fondo se suma a dL_dalphi *después* de que se hayan escrito los gradientes de a , φ , b y β (`backward.cu`:568–571):

$$dL_dalphi += \left(\frac{-T_{\text{final}}}{1 - \alpha} \right) \sum_c \text{bg}_c \cdot \frac{\partial L}{\partial \text{pix}_c}$$

mientras que `dL_d_rho` sí lo incluye (es el “FIX #3” documentado en el código). Es decir: los gradientes del kernel omiten la contribución del color de fondo.

En los runs de este proyecto el término es exactamente cero, porque `white_background` está en `False` y el fondo es negro (`train.py:79`), así que $\text{bg} \cdot \partial L / \partial \text{pix} = 0$. Pero deja de serlo si alguna vez se entrena con `-white_background` (por ejemplo, en los datasets sintéticos tipo Blender): ahí $\partial L / \partial a_n$ quedaría sesgado respecto de la derivada exacta, mientras que $\partial L / \partial \rho$ no. Es una trampa latente, no un bug activo.

7. Fase 4 — Python: restricciones y paso del optimizador

Cada iteración, después de `optimizer.step()`, se aplican tres correcciones en este orden (`train.py:403–456`):

1. **Clamp de β** a $[-4, 2]$ en el parámetro crudo. El techo vive en una sola constante, `BETA_RAW_MIN/MAX`, desde que un `print` desincronizado anunció durante varios runs un experimento que no se estaba corriendo.
2. **Clamp de la escala** sobre el parámetro crudo, no sobre la activación. Mata el “trinquete”: con `torch.clamp` el gradiente no pasa por encima del máximo, así que un `splat` que cruce el techo se congelaría para siempre.
3. **Proyección de a** sobre el conjunto factible.

7.1. La proyección: gradiente proyectado exacto

`project_a_` (`gaussian_model.py:404–437`) implementa la proyección euclídea *exacta* sobre

$$\mathcal{A} = \{a \in \mathbb{R}^3 : a_n \geq 0, \sum a \leq S_{\text{máx}}\},$$

que es convexo (una caja intersecada con un semiespacio):

1. $w = \max(a, 0)$ proyecta sobre la caja. Si $\sum w \leq S_{\max}$, listo: por ser óptimo sobre un conjunto *mayor* que $\mathcal{A}$, también es la proyección sobre $\mathcal{A}$.
2. Si no, la restricción de suma está activa y la proyección es la del simplex $\{a \geq 0, \sum a = S_{\max}\}$: ordenar descendente, encontrar el mayor ρ con $u_\rho - (\text{cumsum}_\rho - S_{\max})/\rho > 0$, restar el umbral θ a todos (Duchi et al., 2008).

Esto no es un clamp heurístico: es **descenso por gradiente proyectado**, con las garantías que eso conlleva. En tres dimensiones el sort es trivial.

Detalle no cosmético: la tolerancia $_A_SUM_TOL = 10^{-6}$. Sin ella, como la propia proyección deja $\sum a$ exactamente en la frontera, en float32 una fracción de las filas queda un epsilon por encima y se lanzaría el sort sobre millones de filas cada iteración sin cambiar nada.

7.2. La doble imposición de $a_n \geq 0$

$a_n \geq 0$ se impone en *dos* sitios: en `project_a_` (sobre el parámetro) y en `get_a` con un `clamp(min=0)` (sobre el forward). El segundo existe porque `render.py`, `metrics.py` y el visor **no pasan por el bucle de entrenamiento**. Los dos tienen que moverse juntos o aparece inconsistencia $\text{train} \leftrightarrow \text{render}$. La cota de la suma, en cambio, no se repite en el forward: como la proyección escribe sobre el parámetro, lo que se guarda en el `.ply` ya es factible.

7.3. La autocalibración de f_1

Con `GABOR_F1` sin definir, f_1 se elige una sola vez en `create_from_pcd` para que el interruptor $f \cdot W = 1$ caiga en el percentil 90 de las escalas iniciales: el 10 % de surfels más grandes desarrolla textura y el resto degenera al tent (`gaussian_model.py:376-402`).

La anchura efectiva no es el radio del soporte, sino

$$W = 2 s \sigma_{\text{env}}(\beta), \quad \sigma_{\text{env}}(\beta) = \frac{\sqrt{\beta+1}}{(\beta+2)\sqrt{\beta+3}} \quad (14)$$

que es la desviación típica de la distribución $\text{Beta}(1, \beta + 1)$, cuya densidad es proporcional a $(1 - x)^\beta$ — es decir, exactamente el perfil de la envolvente leído como distribución de probabilidad. Con el $\beta \approx 2,24$ medido en los runs sale $\sigma_{\text{env}} \approx 0,186$, o sea $W \approx 0,37$ s: **el radio sobreestima el tamaño útil unas tres veces**. Usar el radio en vez de (14) daría un f_1 demasiado bajo y la onda nacería apagada.

De ahí:

$$f_1 = \frac{\text{extent}}{2 s_{p90} \sigma_{\text{env}}}$$

En *flowers* salieron 46,6247 rad/extent.

Trampa operativa. `render.py` llama a `load_ply`, no a `create_from_pcd`, así que **la autocalibración no corre en el render** y f_1 se queda en 0. Hay que pasarle el valor por variable de entorno. El primer render del run 5 se hizo sin ella: el modelo se evaluó con `legacy/norm/f1=0` y con una cota Σa más estrecha que la del entrenamiento, así que `load_ply` *reproyectó el 29,2 % de los splats y mutó el modelo*. Resultado: $-5,07$ dB y dianas en el césped, con el modelo intacto. Hoy `render_server.sh` lee la configuración del log del train y la verifica.

7.4. Tasa de aprendizaje

a se optimiza con `lr = 0.001` (`gaussian_model.py:511`), tres órdenes por debajo de la de posición. No se ha barrido.

8. Fase 5 — Densificación y ciclo de vida

8.1. Herencia en clone y split

a se hereda **tal cual** en las dos rutas de creación (`gaussian_model.py:900` y `:926`):

```
new_a = self._a[selected_pts_mask].repeat(N, 1)  # split
new_a = self._a[selected_pts_mask]                # clone
```

El criterio es el mismo que arregló el trinquete de β : la escala y la opacidad son magnitudes **extensivas** y se reparten entre los hijos ($/0,8N$ y $/N$); a y β son parámetros de **forma**, adimensionales, y no se reparten. Repartir β entre hijos era exactamente el bug que degradaba el kernel a caja tras seis generaciones de split.

8.2. Pero la frecuencia sí se hereda mal (y esto es un hallazgo)

Aquí hay una interacción que no está escrita en ningún sitio del código y que explica la principal patología medida. En el split:

$$s^{\text{hijo}} = \frac{s^{\text{padre}}}{0,8N}, \quad \text{y por (5)} \quad \varphi^{\text{hijo}} = \frac{\varphi^{\text{padre}}}{0,8N}$$

Como a se hereda intacto pero φ cae con la escala, **cada split apaga un poco más la onda**. El hijo conserva la *amplitud* de la modulación del padre pero pierde su *frecuencia efectiva*: ya no le cabe ni medio ciclo dentro de la huella, y $S \rightarrow b \approx \text{constante}$, o sea tent.

Esto es exactamente la deriva medida en los tres runs:

apagados ($f \cdot W < 0,5$)	run5	run6	run7
@2500	88,6 %	73,1 %	43,3 %
@7000	94,3 %	86,0 %	61,0 %
@30000	93,4 %	83,8 %	64,1 %

El salto empeora siempre entre 2500 y 7000, que es la ventana de densificación más intensa. Y explica por qué subir f_1 desplaza la curva entera pero no elimina la pendiente: f_1 es una constante fijada con las escalas *iniciales*, y la densificación mueve el blanco. La única variante viva de esa línea es **recalibrar f_1 durante el run**.

8.3. Serialización

a se guarda en el .ply como a_0 , a_1 , a_2 (solo los tres entrenables: φ y b se recalculan). Al cargar (gaussian_model.py:653–680) hay dos salvaguardas: si el .ply es antiguo y no tiene campos a_* , se cae al init del modo; y siempre se llama a `project_a_`, avisando *solo* si la violación es de magnitud real, para no dar falsos positivos por el redondeo del roundtrip.

Ese aviso, `[A] load_ply: ... PROYECTADOS`, es el que delata el problema de configuración descrito antes.

9. Verificación

`scripts/test_gabor_atomo.py` comprueba tres cosas contra el rasterizador *real*, no contra una reimplementación:

1. **El punto neutro.** Con $a = 0$, los modos `radial` y `dir` dan la misma imagen y coinciden con $(1 - r)^\beta$ bit a bit.
2. **Los gradientes** de a_n , φ , b , β y las escalas, contra diferencias finitas: medianas de error del 0,2 al 2,9 %.
3. **No regresión:** el modo `legacy` sigue dando lo de antes, con 2,5 % en `scaling` como control.

Los porcentajes parecen altos para una comprobación de gradientes, pero son los esperables en float32 con un rasterizador con early-outs discontinuos ($\alpha < 1/255$, $T < 1e-4$): una perturbación finita puede cruzar un umbral y cambiar el conjunto de splats que contribuyen.

10. Qué se midió

10.1. El rediseño funciona; la onda no compra

run	PSNR	SSIM	LPIPS
run2 legacy	20,1602	0,5537	0,3900
run67 tent	20,6684	0,5811	0,3675
run5 átomo $1\times$	20,6898	0,5759	0,3724
run6 átomo $2\times$	20,5999	0,5763	0,3716
run7 átomo $4\times$	20,7167	0,5758	0,3720

Dos lecturas, las dos importantes:

- Contra el **legacy**, el átomo gana en las tres métricas (+0,53 dB). El rediseño hace lo que prometía: *aprender la forma ya no resta*.
- Contra el **tent**, empate dentro del ruido. Y el sweep de f_1 demostró que no es porque la onda esté apagada: con f_1 a $4\times$ se encendieron 5,9 veces más splats, con más amplitud (Σa medio 0,2909 $\rightarrow$ 0,3429) y menos kernels planos (21,4 $\rightarrow$ 12,9 %), y las métricas no se movieron.

Conclusión: en *flowers*, **todo el beneficio del átomo viene de la envolvente** (el pedestal que hace del tent el punto neutro), no de la oscilación. Lo cual tiene sentido: el césped y el follaje son textura estocástica de alta frecuencia sin estructura oscilatoria coherente, y un armónico aprendido no tiene ahí nada que encajar. Está pendiente comprobarlo en escenas con estructura (bonsai, kitchen).

10.2. La firma que sí se repite

En los cuatro runs del átomo, sin excepción: a_1 baja y a_3 sube durante el entrenamiento, hasta que en el run 7 se cruzan ($a_3 = 0,1237 > a_1 = 0,1153$). El kernel gasta el presupuesto en el armónico más alto — espectro creciente,

justo lo contrario de una envolvente. Es la señal con más contenido que queda sin explicar, y la que apunta a la siguiente palanca: forzar $a_1 > a_2 > a_3$.

11. Invariantes

Lo que no se puede romper sin romper el kernel:

1. **Una sola definición.** K y sus derivadas viven en `gabor_kernel.h`. Forward y backward lo incluyen los dos.
2. $a = 0$ **debe dar la tent exacta.** Es el control experimental. Si un cambio lo rompe, el baseline deja de ser comparable.
3. **El clamp de α es el mismo en los dos pases.**
4. **Las variables de entorno del render deben coincidir con las del train,** f_1 incluido. `render_server.sh` lo verifica solo.
5. **Cambiar el layout del bloque a obliga a recompilar** el rasterizer. Cambiar γ , f_1 o κ no.
6. **Los diagnósticos se leen por histograma,** nunca por el máximo. El proyecto tiene ya varios casos de un dial que parecía activo y era un no-op.

A. Símbolos

r	distancia normalizada, $r = \sqrt{\rho}$, $r < 1$
u, v	coordenadas locales del surfel
ρ_{3d}	$u^2 + v^2$, geometría real
ρ_{2d}	filtro paso-bajo de pantalla
E	envolvente $(1 - r)^\beta$
S	onda, ecuación (2)
b	pedestal, ecuación (3)
φ	frecuencia base efectiva
w_n	$(2n - 1)\varphi$
t	argumento de la onda: r o u
γ	suelo del pedestal (0,3)
f_1	frecuencia en rad por unidad de extent
κ	dial de contraste (1,0)
β	exponente de la envolvente
W	anchura efectiva, ecuación (14)

B. Mapa del código

kernel y derivadas	<code>gabor_kernel.h</code>
copia por-Gaussiana	<code>forward.cu:193</code>
evaluación	<code>forward.cu:474–503</code>
recomputación	<code>backward.cu:333–369</code>
5 gradientes	<code>backward.cu:466–490</code>
cadena de ρ	<code>backward.cu:551–564</code>
φ y b	<code>gaussian_model.py:352</code>
autocalibración	<code>gaussian_model.py:376</code>
proyección	<code>gaussian_model.py:404</code>
init	<code>gaussian_model.py:298</code>
herencia	<code>gaussian_model.py:900, 926</code>
paso del optimizador	<code>train.py:403–456</code>
diagnósticos	<code>train.py:542–600</code>
verificación	<code>scripts/test_gabor_atomo.py</code>

C. Documentos relacionados

- `docs/rediseno_kernel_gabor_adagar.html` — el rediseño con las 14 figuras y los Monte Carlo del conjunto factible.
- `docs/analisis_run2_y_codigo_completo.html` — el diagnóstico del kernel legacy.
- `docs/render_y_env_vars.html` — el pipeline de render y la tabla de variables de entorno.
- `CLAUDE.md` — estado actual, baselines y palancas abiertas.